Procedimientos almacenados

Dónde estamos

- La función devuelve un valor y no debería cambiar nada. El procedimiento existe justamente para lo contrario: ejecutar una acción sobre los datos.
- Con el procedimiento entran al curso tres temas que ya no son de sintaxis: transacciones, privilegios y manejo de errores.

Qué vamos a hacer

- Escribir procedimientos con validación, manejo de excepciones y un contrato claro de parámetros.
- Decidir quién confirma la transacción y por qué casi nunca debe hacerlo el procedimiento.
- Agrupar el código en paquetes y procesar volúmenes con BULK COLLECT y FORALL.

Recorrido

- Sintaxis y modos de parámetro. Excepciones y trazabilidad. Transacciones y transacción autónoma.
- Paquetes, sobrecarga, procesamiento masivo, privilegios y dependencias. Taller guiado con el script 03.

Lo que hay que tener fresco

La transacción no empieza donde usted cree

- Empieza con la primera sentencia DML de la sesión y termina con COMMIT o ROLLBACK, no con el fin del procedimiento.
- Si el procedimiento hace tres UPDATE y falla el tercero, los dos primeros siguen vivos en la transacción hasta que alguien decida.

Excepción no atrapada

- Se propaga al bloque que invocó y deshace el trabajo de la sentencia que falló, no el de toda la transacción.
- Por eso el procedimiento debe decidir de forma explícita qué hace ante el error: devolverlo, registrarlo o revertir hasta un savepoint.

Privilegios

- Para invocar un procedimiento de otro esquema se necesita EXECUTE sobre ese objeto, no privilegios sobre las tablas que toca.
- Ese es el mecanismo estándar para exponer operaciones sin abrir las tablas.

Procedimiento o función: la decisión práctica

Pregunta	Si la respuesta es sí	Unidad indicada
¿Modifica datos o produce un efecto en el sistema?	Registra, actualiza, envía, borra	Procedimiento
¿El resultado es un valor que quiero usar en una expresión?	Se asigna, se compara, se muestra	Función
¿Necesito invocarlo desde una consulta SQL?	Aparece en SELECT, WHERE u ORDER BY	Función, solo con parámetros IN
¿Devuelve varios datos a la vez?	Tres o cuatro valores de salida	Procedimiento con parámetros OUT, o un registro

Regla que resuelve el 90 por ciento de los casos
Sustantivo, función. Verbo de acción, procedimiento. Si el nombre honesto de la unidad es calcular_y_registrar, hay dos unidades escondidas en una.

Anatomía de CREATE PROCEDURE

```
CREATE OR REPLACE PROCEDURE pr_registrar_matricula (
  p_id_estudiante IN  pr_estudiante.id_estudiante%TYPE,
  p_id_asignatura IN  pr_asignatura.id_asignatura%TYPE,
  p_id_matricula  OUT pr_matricula.id_matricula%TYPE)
IS
  v_cupo NUMBER;
BEGIN
  -- validar, actuar, informar: siempre en ese orden
  NULL;
END pr_registrar_matricula;
/
```

Dos decisiones de diseño visibles en la firma

Los tipos se declaran con %TYPE, de modo que el procedimiento siga los cambios de las columnas. Y el identificador generado sale por un parámetro OUT: quien invoca no tiene que volver a consultarlo.

Modos de parámetro

Modo	Dentro del procedimiento	Al invocar	Uso
IN (valor por omisión)	Solo lectura, es una constante	Acepta literal, variable o expresión	Datos de entrada
OUT	Empieza en NULL y debe asignarse	Exige una variable, no un literal	Resultado de la operación
IN OUT	Entra con valor y sale modificado	Exige una variable	Transformar un valor recibido

```
-- NOCOPY sugiere pasar por referencia en lugar de copiar el valor
PROCEDURE pr_procesar (p_datos IN OUT NOCOPY t_lista_notas)
```

Cuidado con NOCOPY
Es una sugerencia al compilador, no una orden. Y si el procedimiento falla a mitad de camino, el parámetro puede quedar modificado parcialmente, porque no hubo copia que descartar.

Un procedimiento completo: validar, actuar, informar

```
BEGIN
  SELECT cupo_disponible INTO v_cupo
    FROM pr_asignatura WHERE id_asignatura = p_id_asignatura FOR UPDATE;

  IF v_cupo <= 0 THEN
     RAISE_APPLICATION_ERROR(-20010, 'Sin cupo en la asignatura ' || p_id_asignatura);
  END IF;

  INSERT INTO pr_matricula (id_estudiante, id_asignatura, fecha)
       VALUES (p_id_estudiante, p_id_asignatura, SYSDATE)
    RETURNING id_matricula INTO p_id_matricula;

  UPDATE pr_asignatura SET cupo_disponible = cupo_disponible - 1
   WHERE id_asignatura = p_id_asignatura;
END;
```

FOR UPDATE bloquea la fila del cupo hasta el final de la transacción: sin ese bloqueo, dos sesiones simultáneas pueden pasarse del cupo.

Cómo se invoca

```
-- desde SQL*Plus o SQL Developer
EXEC pr_registrar_matricula(101, 55, :v_id);

-- desde un bloque, con notación nombrada
BEGIN
  pr_registrar_matricula(
    p_id_estudiante => 101,
    p_id_asignatura => 55,
    p_id_matricula  => v_id);
END;
```

```
-- otorgar el permiso de ejecución
GRANT EXECUTE ON pr_registrar_matricula TO app_user;

-- invocarlo desde otro esquema
BEGIN
  admin.pr_registrar_matricula(101, 55, v_id);
END;
/
```

Por qué la notación nombrada gana en código real

Una firma con cinco parámetros del mismo tipo es un accidente esperando ocurrir. Con notación nombrada, el error de orden desaparece y la invocación se lee sin abrir la definición.

Excepciones: informar sin perder la traza

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20011, 'Estudiante inexistente: ' || p_id_estudiante);
  WHEN OTHERS THEN
    INSERT INTO pr_log_error (fecha, unidad, codigo, mensaje, traza)
         VALUES (SYSTIMESTAMP, 'pr_registrar_matricula', SQLCODE, SQLERRM,
                 DBMS_UTILITY.FORMAT_ERROR_BACKTRACE);
    RAISE;   -- vuelve a levantar: quien invoca decide qué hacer
END;
```

La diferencia entre RAISE y no hacer nada

RAISE conserva el error original y su traza. Un WHEN OTHERS que solo registra y termina deja al llamador creyendo que todo salió bien: es la causa más común de datos inconsistentes sin ningún mensaje de error.

Quién confirma la transacción

```
-- dentro del procedimiento: control parcial con savepoint
SAVEPOINT antes_de_matricula;
  -- ... operaciones ...
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK TO antes_de_matricula;   -- deshace solo lo propio
    RAISE;
```

Escenario	Quién debe hacer COMMIT
Procedimiento invocado desde una aplicación	La aplicación, al cerrar la unidad de trabajo completa
Procedimiento que es una unidad de trabajo completa por sí mismo	El procedimiento, y debe documentarse en su cabecera
Procedimiento invocado desde otro procedimiento	Ninguno de los dos: el que abrió la transacción
Registro de auditoría que debe sobrevivir a un ROLLBACK	Una transacción autónoma, en la lámina siguiente

PRAGMA AUTONOMOUS_TRANSACTION

Una transacción independiente dentro de otra: confirma lo suyo sin tocar la del llamador

```
CREATE OR REPLACE PROCEDURE pr_log (p_unidad VARCHAR2, p_mensaje VARCHAR2)
IS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  INSERT INTO pr_log_error (fecha, unidad, mensaje)
       VALUES (SYSTIMESTAMP, p_unidad, p_mensaje);
  COMMIT;   -- obligatorio: la transacción autónoma debe cerrarse
END;
/
```

Uso legítimo y uso peligroso

Legítimo: auditoría y bitácoras que deben persistir aunque la operación principal se revierta. Peligroso: usarla para esquivar un bloqueo o una tabla mutante, porque deja los datos en un estado que nadie puede explicar.

Paquetes: especificación y cuerpo

```sql
CREATE OR REPLACE PACKAGE pk_matricula IS
  e_sin_cupo EXCEPTION;
  PRAGMA EXCEPTION_INIT(e_sin_cupo, -20010);

  PROCEDURE registrar(p_est NUMBER, p_asi NUMBER,
                      p_id OUT NUMBER);
  PROCEDURE anular(p_id NUMBER);
END pk_matricula;
/
```

```sql
CREATE OR REPLACE PACKAGE BODY pk_matricula IS
  -- privado: no existe fuera del paquete
  FUNCTION hay_cupo(p_asi NUMBER) RETURN BOOLEAN IS
  BEGIN RETURN TRUE; END;

  PROCEDURE registrar(p_est NUMBER, p_asi NUMBER,
                      p_id OUT NUMBER) IS
  BEGIN NULL; END;
END pk_matricula;
```

Tres ventajas concretas

La dependencia de quien invoca es con la especificación, así que cambiar el cuerpo no invalida el código cliente. Se permite la sobrecarga. Y el tipo BOOLEAN es válido entre unidades PL/SQL, aunque no lo sea en SQL.

Procesamiento masivo: BULK COLLECT y FORALL

```
DECLARE
  TYPE t_ids IS TABLE OF pr_matricula.id_matricula%TYPE;
  v_ids t_ids;
  CURSOR c IS SELECT id_matricula FROM pr_matricula WHERE estado = 'PEN';
BEGIN
  OPEN c;
  LOOP
    FETCH c BULK COLLECT INTO v_ids LIMIT 500;   -- acota la memoria de sesión
    EXIT WHEN v_ids.COUNT = 0;
    FORALL k IN 1..v_ids.COUNT SAVE EXCEPTIONS
      UPDATE pr_matricula SET estado = 'ACT' WHERE id_matricula = v_ids(k);
  END LOOP;
  CLOSE c;
END;
```

SAVE EXCEPTIONS permite terminar el lote y revisar después los fallos en SQL%BULK_EXCEPTIONS, en vez de abortar en el primer error.

Privilegios, dependencias y estado

```
GRANT EXECUTE ON pk_matricula TO rol_academico;      -- permiso de uso

SELECT object_name, object_type, status               -- objetos invalidados
  FROM user_objects WHERE status = 'INVALID';

SELECT name, type, referenced_name                    -- de qué depende
  FROM user_dependencies WHERE referenced_name = 'PR_MATRICULA';

ALTER PACKAGE pk_matricula COMPILE BODY;              -- recompilar el cuerpo
```

Qué pasa en un despliegue

Alterar una tabla invalida todo lo que depende de ella. Oracle recompila en la siguiente invocación, pero si esa recompilación falla, el error aparece en producción. Revise USER_OBJECTS antes de dar por cerrado el despliegue.

Errores frecuentes con procedimientos

Síntoma	Causa típica	Corrección
PLS-00306: número o tipo de argumentos erróneo	Orden de parámetros cambiado en la invocación	Usar notación nombrada
El parámetro OUT llega nulo	Una ruta del código no le asigna valor	Asignarlo al inicio o cubrir todas las ramas
Los cambios no aparecen para otras sesiones	Nadie ejecutó COMMIT	Definir y documentar quién cierra la transacción
ORA-04068: estado del paquete descartado	Se recompiló el paquete con sesiones activas	Desplegar en ventana, o evitar estado en el paquete
El error se pierde y los datos quedan a medias	WHEN OTHERS que registra y no vuelve a levantar	Agregar RAISE después de registrar

Cómo trabajar el script 03 paso a paso

Antes de ejecutar

- Abra 03_procedimientos_plsql19c.sql con un usuario que tenga CREATE TABLE, CREATE PROCEDURE y CREATE SEQUENCE.
- Ejecute SET SERVEROUTPUT ON. El script crea su propio esquema de práctica con el prefijo PR_ y no depende de los otros módulos.

Durante la práctica

- Cada paso indica qué se va a hacer, qué debe observar y cuál es el resultado esperado; ejecútelos en orden.
- En el paso de concurrencia se le pide abrir una segunda sesión: ahí se ve el bloqueo del FOR UPDATE en vivo.
- El paso de transacción autónoma exige mirar la tabla de log después de un ROLLBACK. No lo salte: es el punto de la sesión.

Al terminar

- Resuelva los cinco ejercicios propuestos, cada uno con pista y criterio de logrado.
- Ejecute la limpieza final solo cuando haya terminado toda la práctica.

Lo que hay que llevarse

Del criterio

- El procedimiento actúa; la función calcula. Un nombre con y en la mitad suele esconder dos unidades.
- Quien abre la transacción la cierra. Un COMMIT escondido dentro de un procedimiento reutilizable es una bomba de tiempo.
- Toda excepción atrapada debe terminar en una decisión escrita: relanzarla, traducirla o revertir hasta un savepoint.

Del oficio

- Notación nombrada al invocar, %TYPE al declarar, paquete para agrupar. Tres hábitos que ahorran horas de mantenimiento.
- Antes de escribir un bucle que hace DML fila a fila, pregúntese si el problema se resuelve con una sola sentencia SQL.

Lecturas para llegar preparado a la próxima sesión

- PL/SQL Language Reference 19c, capítulos PL/SQL Subprograms y PL/SQL Packages.
- Database Development Guide 19c, secciones sobre manejo de errores y procesamiento masivo.
- docs.oracle.com/en/database/oracle/oracle-database/19/lnpls